

Map-Reduce and Sharded Collections

NOTE

Aggregation Pipeline as an Alternative to Map-Reduce

Starting in MongoDB 5.0, map-reduce is deprecated:

- Instead of map-reduce, you should use an aggregation pipeline. Aggregation pipelines provide better performance and usability than map-reduce.
- You can rewrite map-reduce operations using aggregation pipeline stages, such as `$group`, `$merge`, and others.
- For map-reduce operations that require custom functionality, you can use the `$accumulator` and `$function` aggregation operators. You can use those operators to define custom aggregation expressions in JavaScript.

For examples of aggregation pipeline alternatives to map-reduce, see:

- [Map-Reduce to Aggregation Pipeline](#)
- [Map-Reduce Examples](#)

Map-reduce supports operations on sharded collections, both as an input and as an output. This section describes the behaviors of `mapReduce` specific to sharded collections.

Sharded Collection as Input

When using sharded collection as the input for a map-reduce operation, `mongos` will automatically dispatch the map-reduce job to each shard in parallel. There is no special option required. `mongos` will wait for jobs on all shards to finish.

Sharded Collection as Output

If the `out` field for `mapReduce` has the `sharded` value, MongoDB shards the output collection using the `_id` field as the shard key.

To output to a sharded collection:

- If the output collection does not exist, create the sharded collection first.
- If the output collection already exists but is not sharded, map-reduce fails.
- For a new or an empty sharded collection, MongoDB uses the results of the first stage of the map-reduce operation to create the initial chunks distributed among the shards.
- `mongos` dispatches, in parallel, a map-reduce post-processing job to every shard that owns a chunk. During the post-processing, each shard will pull the results for its own chunks from the other shards, run the final reduce/finalize, and write locally to the output collection.

NOTE

- During later map-reduce jobs, MongoDB splits chunks as needed.
- Balancing of chunks for the output collection is automatically prevented during post-processing to avoid concurrency issues.

